

Databricks integration recommendation for CHEC Local UITI_VANO Interpreter

Executive summary

The recommended path is a **hybrid architecture**:

- **Databricks** should own data, curated tables, feature engineering, model retraining, model registry, serving endpoints, and operational dashboards.
- **This repository + Pi / el Gentleman** should continue owning `/skill:report`, the role-based agent flow (**historical**, **inference**, **expert-alignment**), validation contracts, and local HTML report generation.

A full migration of Pi skills, subagents, Gentle AI harnesses, and Engram-style memory into Databricks is technically possible only through a substantial platform rewrite. It is **not** the best first move.

Why this recommendation fits this project

This repository is not only a model-consumption app. Its value is split across four layers:

1. **Deterministic Python** builds context and validates outputs.
2. **Agent-native reasoning** is executed by the coding-agent runtime, not by Python LLM SDK calls.
3. **/skill:report orchestration** coordinates multiple roles and renders a final HTML artifact.
4. **Harness discipline** from Pi / el Gentleman / Gentle AI controls skills, subagents, review flow, and execution behavior.

Because of that design, Databricks is a strong fit for the **data and ML platform** layer, but not a natural replacement for the full **agent runtime and harness** layer.

Current architecture observations from the repo

Verified from project documentation:

- `docs/agents-guide.md` describes a layered architecture where deterministic Python tools build context, validate responses, and isolate agent-facing CLIs from the core logic.
- `.pi/skills/report/SKILL.md` confirms that the Pi runtime currently orchestrates report generation through `report_pipeline.py` and `report_contract.py`.
- The project explicitly avoids external Python-side LLM API calls; the invoking coding-agent runtime performs the reasoning.

This strongly suggests that the safest evolution is to replace the **data/model backend**, not the **agent orchestration runtime**.

Option analysis

Option 1 — Hybrid integration (recommended)

Shape

Databricks - Delta / Unity Catalog tables for raw and curated datasets - Feature engineering pipelines - Lakeflow Jobs for scheduled retraining - MLflow tracking and model registry - Databricks Model Serving endpoints - Databricks SQL / Lakeview dashboards

Pi / local runtime - `/skill:report - report_pipeline.py - agent_tools/*` - role contracts and validators - local HTML rendering - Gentle AI / Pi harnesses and memory tooling

Integration patterns

Pattern A: external Pi runtime, Databricks as backend The current repo runs outside Databricks and reads: - Delta/SQL data via Databricks connectors - model metadata via MLflow - predictions or explanations via Model Serving endpoints

Pros - Minimal disruption to current agent workflow - Preserves skills, sub-agents, and harness discipline - Fastest path to value

Cons - Cross-environment credentials and networking - Runtime split between local agent environment and cloud data platform

Pattern B: deterministic pipeline pieces run in Databricks jobs Move some non-agent Python steps into Databricks Jobs while keeping the agent reasoning in Pi.

Pros - Better operational scheduling - Stronger governance for repeatable ETL/training jobs

Cons - The agent-native step still remains outside Databricks - More orchestration complexity than Pattern A

Recommendation inside Option 1

Start with **Pattern A**. It preserves the current user workflow while centralizing the expensive and operational parts in Databricks.

Option 2 — Semi-migration

Databricks owns: - data - training - inference services - explainability outputs - dashboards

The local Pi environment still owns: - report orchestration - all agents and skills
- JSON validation - HTML report rendering

This is a valid target state if the main objective is platform centralization without changing how analysts trigger reports.

Option 3 — Full migration of agent runtime into Databricks

This would require moving or rebuilding: - Pi skills - markdown subagents - Gentle AI harness behaviors - Engram-like memory behavior - command endpoints such as `/skill:report` - review and orchestration discipline now enforced by Pi/el Gentleman

Why this is not recommended as the first move

1. **Databricks is not the native runtime for Pi harnesses.**
2. **Interactive skill-driven command workflows do not map cleanly to Jobs or notebooks.**
3. **Subagent orchestration and review receipts are platform-specific behaviors, not generic Python code.**
4. **Memory and harness guarantees would need replacement implementations.**
5. **This becomes a platform rewrite, not a backend integration.**

Option 4 — Rebuild on Databricks Mosaic AI Agents

Databricks now supports agent-oriented capabilities such as tool calling, agent deployment, and vector/AI search.

However, using Mosaic AI as the new home for this solution would still require rebuilding: - role orchestration - validation boundaries - skill system behavior - memory semantics - report-generation workflow

This is strategically possible, but it should be treated as a **new platform initiative**, not a straightforward migration.

Target architecture

Layer 1 — Databricks platform

- Unity Catalog + Delta tables
- curated training and reporting datasets
- Lakeflow Jobs for retraining and batch refreshes
- MLflow for experiments and registry
- Model Serving for online/batch prediction APIs
- SQL/Lakeview dashboards for operational visibility

Layer 2 — Integration adapters in this repo

Add explicit adapters such as: - `src/chec_local_interpreter/databricks_data_source.py`
- `src/chec_local_interpreter/databricks_model_client.py` - `src/chec_local_interpreter/databricks`

Responsibilities: - read filtered circuit/date data from Databricks - fetch model versions and metadata from MLflow - call serving endpoints for predictions/explanations when needed - normalize external results into the existing context contracts

Layer 3 — Existing local agent orchestration

Keep unchanged initially: - `report_pipeline.py` - `report_contract.py` - `agent_tools/*` - role skills and validators - Pi / el Gentleman operator flow

Layer 4 — Outputs

- local HTML report output
- optional published artifact store
- links back to Databricks dashboards or lineage metadata

Migration roadmap

Phase 1 — Backend connection without workflow breakage

- Keep the current repo and Pi workflow intact.
- Move datasets and model assets to Databricks.
- Add data/model adapters in this repo.
- Ensure `/skill:report` still feels identical to the operator.

Phase 2 — Production-grade ML operations

- Scheduled retraining in Databricks Jobs
- Model registration and promotion via MLflow
- Model Serving endpoints for prediction and explanation access
- Dashboards for monitoring model and data refresh status

Phase 3 — Selective cloud execution

- Move deterministic, non-interactive preprocessing to Databricks jobs where valuable.
- Keep agent reasoning and final report orchestration in Pi unless there is a strong reason to replace it.

Phase 4 — Strategic reevaluation

- Only if the organization wants to retire Pi as runtime:
 - assess Mosaic AI Agents / Apps

- redesign memory, skills, and orchestration semantics
- treat as a separate architecture program

Key risks

Risk	Why it matters	Mitigation
Credential and network complexity	Local Pi runtime must securely reach Databricks assets	Use service principals, secret scopes, and narrow permissions
Data contract drift	Existing report contexts may assume local schemas	Add adapter-level schema normalization and contract tests
Model/report traceability	Reports must identify which model/data snapshot they used	Persist model version, serving endpoint revision, and table snapshot metadata in run artifacts
Latency	Remote reads or serving calls can slow report generation	Cache immutable intermediate artifacts per run and batch where possible
Platform rewrite risk	Full migration would duplicate harness behavior	Avoid full migration until backend integration is stable

What should be avoided

- Treating a Databricks notebook as a drop-in replacement for `/skill:report`
- Rewriting Pi/Gentle AI harness behavior before validating the backend integration
- Coupling training, serving, dashboards, and agent orchestration into one runtime too early
- Starting with a full Mosaic AI rewrite before proving the business value of a hybrid architecture

Final recommendation

Adopt **Databricks as the system of record for data and ML operations**, while preserving **Pi / el Gentleman as the controlled agent runtime** for report generation and orchestration.

This gives the organization: - scalable data and ML operations - governed re-training and serving - dashboards and operational visibility - minimal disruption to the current role-based analytical workflow

A full migration of agents, skills, harnesses, and memory to Databricks should be considered only after the hybrid model is stable and there is a deliberate decision to replace Pi as the runtime.

Sources consulted

- Databricks Model Serving: <https://docs.databricks.com/aws/en/machine-learning/model-serving>
- Databricks Feature Store: <https://docs.databricks.com/aws/en/machine-learning/feature-store/>
- Lakeflow Jobs: <https://docs.databricks.com/aws/en/jobs>
- Databricks Apps: <https://docs.databricks.com/aws/en/dev-tools/databricks-apps>
- Build AI agents on Databricks: <https://docs.databricks.com/aws/en/agents/>
- Declarative Automation Bundles / workspace collaboration: <https://docs.databricks.com/aws/en/dev-tools/bundles/workspace>
- Workspace files limitations/behavior: <https://docs.databricks.com/aws/en/files/workspace>
- Project architecture: `docs/agents-guide.md`
- Pi report skill contract: `.pi/skills/report/SKILL.md`